

ease the task of creating these common kinds of views, Django includes what it calls Class-Based Generic views.

Generic views called `ListView` and `DetailView` (there are others) are available for developers to base their class-based views on so that they can focus on the bits that are different, such as the templates.

We already saw the `DetailView` in action as it rendered an article page using just two lines of code. If we wanted to use the `ListView` generic view for rendering our article list page, the procedure is exactly the same. The only code we will need is the following:

```
• class ArticleList(ListView):
•     model = Article
```

This view will automatically fetch a list of articles, load the template from the conventional location (`cms/articles_list.html` derived from `<app name>/<model name>_list.html`) supply it the list of articles to render it and then return that data to display in the browser. You could just make a copy of the above and put in `Gallery` instead of `Article` and have a functioning `Gallery` view!

You may be wondering, what's left to do now? Class-based generic views have solved all our problems, so let's just move along to the next chapter. Unfortunately, we have cheated till now and you will soon understand why the above code can't work for the article list, or even the article.

We have conveniently ignored till now the fact the we only want to display a list of published articles. The views above have no way of knowing that and end up showing all articles, even the unpublished ones. This is of course bad, but the good news is that the solution is very simple and is as small as the above code sample.

Our solution to only display articles have are published is as follows:

```
• class ArticleList(ListView):
•     queryset = Article.objects.filter(
•         publish__status=True
```

Notice that instead of specifying a model, we are directly specifying the list of objects to use in for this `ListView`. This is still only two lines of code, works the same way as before, with the same template.

We can similarly use a queryset with our `ArticleDetail` view. If we continue to use the old `ArticleDetail` class that specifies a model instead of a queryset, it will be possible for people to open an unpublished article if they know the slug even if the article is unlisted. It will work like unlisted videos on YouTube, where knowing the URL lets you